

Incluir as bibliotecas necessárias para o programa.

```
# -*- coding: utf-8 -*-
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
from google.colab import drive
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import classification_report, confusion_matrix

↳ /usr/local/lib/python3.6/dist-packages/statsmodels/tools/_testing.py:19: FutureWarning
import pandas.util.testing as tm
```

Definicao do nome das colunas da base de dados.

```
# define column names
names = [
    'sepal_length',
    'sepal_width',
    'petal_length',
    'petal_width',
    'class',
]
```

Leitura da base de dados 'iris.data' utilizando a biblioteca 'pandas'

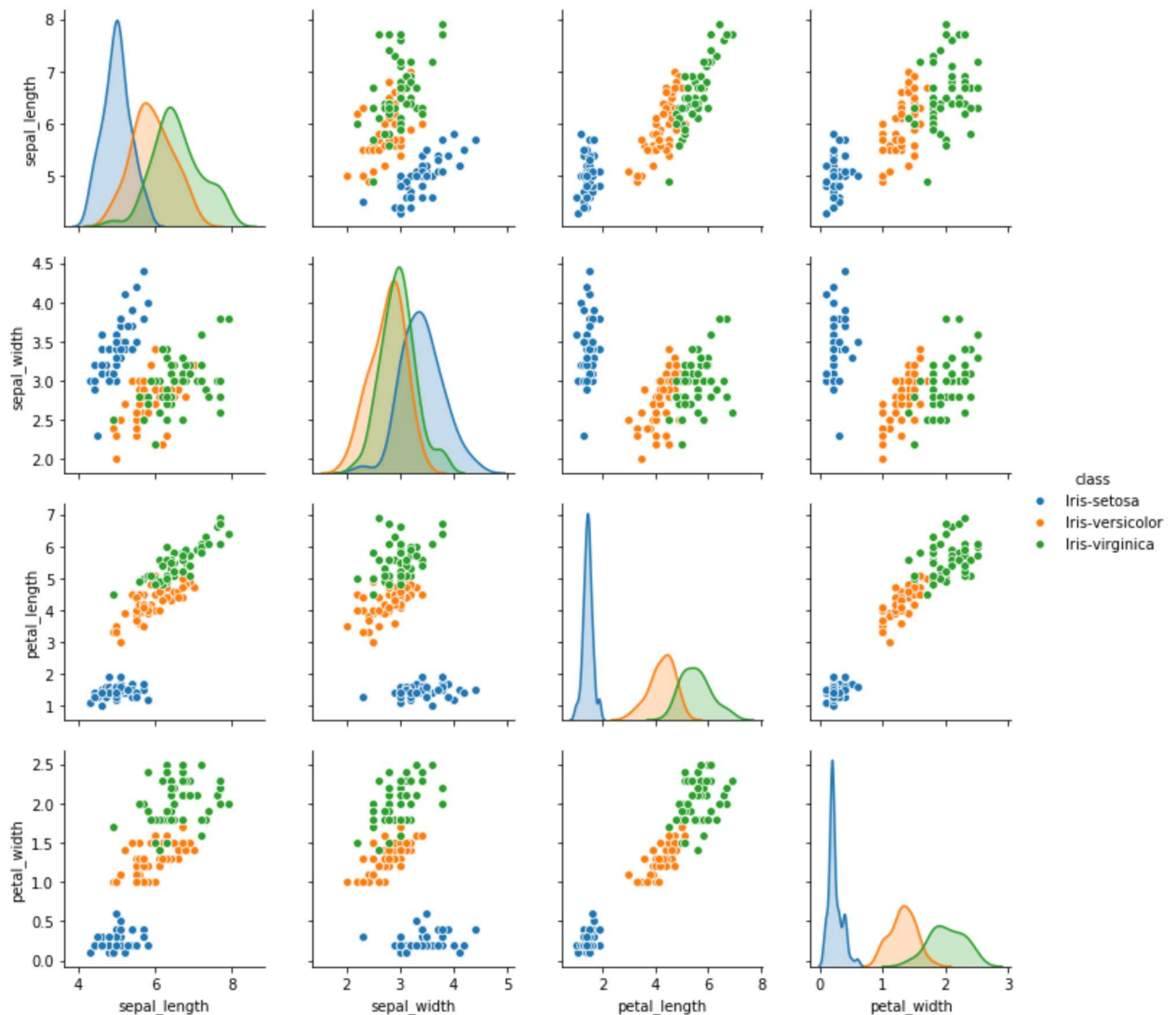
```
# load training data
drive.mount('/content/gdrive')
df = pd.read_csv('/content/gdrive/My Drive/Colab Notebooks/iris.data', header=None, names=
df.head()
```

↳

Go to this URL in a browser: https://accounts.google.com/o/oauth2/auth?client_id=9473

```
sns.pairplot(df,hue="class")
```

```
<seaborn.axisgrid.PairGrid at 0x7f4a2b4e9470>
```



Divide a base de dados em conjunto de treinamento 'X' e label/rotulos 'y'

```
# create design matrix X and target vector y
X = np.array(df.iloc[:, 0:4]) # end index is exclusive
y = np.array(df['class'])    # another way of indexing a pandas df
```

Divide a base de dados em conjunto de treinamento e conjunto de teste. Os respectivos rotulos tambem sao divididos. O parametro 'random_state' recebe um numero (semente) que determina

a saída aleatoria. Caso queira repetir o mesmo conjunto, mantenha a mesma semente. Caso queira um conjunto de dados diferente, eh necessario mudar a semente.

```
# split into train and test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33, random_state=42)
```

Cria um classificador KNN. O objeto/variavel 'knn' eh o modelo que sera treinado e realizara previsoes.

```
# instantiate learning model (k = 3)
knn = KNeighborsClassifier(n_neighbors=3)
```

Faz o 'treinamento' do modelo, que eh chamado de 'ajuste (fit)'. Nesse caso, o conjunto de treinamento 'X_train' e seus respectivos rotulos 'y_train' sao utilizados.

```
# fitting the model
knn.fit(X_train, y_train)

➞ KNeighborsClassifier(algorithm='auto', leaf_size=30, metric='minkowski',
                        metric_params=None, n_jobs=None, n_neighbors=3, p=2,
                        weights='uniform')
```

Apos o modelo ter sido treinado, ele esta pronto para realizar uma 'previsao'. Nesse caso, utiliza o conjunto de test 'X_test' para realizar a previsao.

```
# predict the response
y_pred = knn.predict(X_test)
```

Avalia a 'acuracia' do modelo, comparando a saída gerada pelo modelo 'y_pred' com o rotulo 'y_test' do conjunto de treinamento.

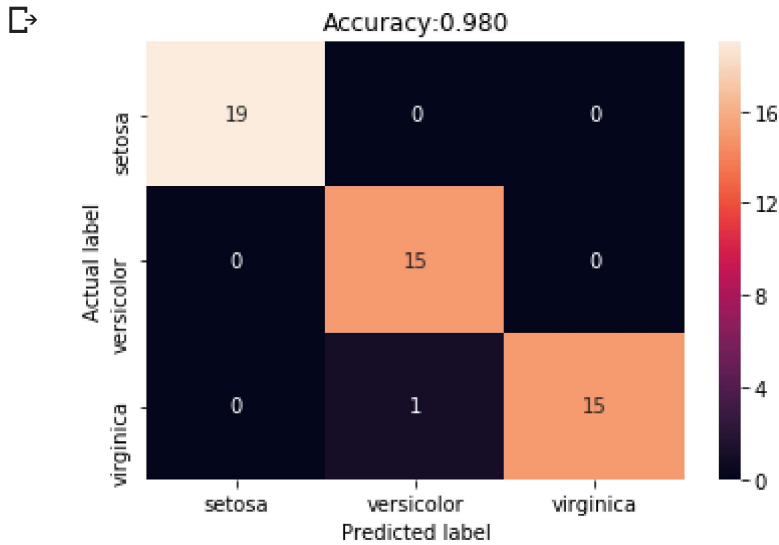
```
# evaluate accuracy
print("accuracy: {}".format(accuracy_score(y_test, y_pred)))
```

```
➞ accuracy: 0.98
```

```
# Creates a confusion matrix
cm = confusion_matrix(y_test, y_pred)
# Transform to df for easier plotting
cm_df = pd.DataFrame(cm,
                      index = ['setosa', 'versicolor', 'virginica'],
                      columns = ['setosa', 'versicolor', 'virginica'])
```

```
sns.heatmap(cm_df, annot=True)
plt.title('Accuracy: {0:.3f}'.format(accuracy_score(y_test, y_pred)))
```

```
plt.ylabel('Actual label')
plt.xlabel('Predicted label')
plt.show()
```



```
from sklearn.metrics import classification_report, confusion_matrix
print('Confusion matrix:')
print(confusion_matrix(y_test, y_pred))
print('\nClassification report:')
print(classification_report(y_test, y_pred))
```

Confusion matrix:

```
[[19  0  0]
 [ 0 15  0]
 [ 0  1 15]]
```

Classification report:

	precision	recall	f1-score	support
Iris-setosa	1.00	1.00	1.00	19
Iris-versicolor	0.94	1.00	0.97	15
Iris-virginica	1.00	0.94	0.97	16
accuracy			0.98	50
macro avg	0.98	0.98	0.98	50
weighted avg	0.98	0.98	0.98	50

